

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS
LABORATORY WORK #11 - PART 1

Behavioural Control with Finite-State Machines: Button-LED

Author:

Sava LUCHIAN
std. gr. FAF-233

Verified:

Alexei MARTINIUC

Chisinau 2026

Technical Task

Purpose of the Laboratory

The purpose of this laboratory is to implement a behavioural embedded-control application using a finite-state machine (FSM). According to the reference condition document, Part 1 requires a Button-LED controller with two states: LED OFF and LED ON. The transition between states occurs on each valid button press. Software debouncing is required to reject unwanted mechanical contact fluctuations, and the current state must be displayed through STDIO/serial or LCD.

Objectives

1. Implement a modular finite-state machine with two states: **OFF** and **ON**.
2. Toggle the LED state only after a valid debounced button press.
3. Report the current FSM state through `printf()` using STDIO.
4. Display the state on a 16x2 LCD.
5. Keep the implementation compatible with Arduino Uno, PlatformIO, and Wokwi.

Individual Task

Implement Part 1 only: **Button-LED behavioural control using finite-state machines**. The button is connected to D12 using `INPUT_PULLUP`; the LED is connected to D9 through a $220\ \Omega$ resistor. The current LED/FSM state is shown on LCD and reported periodically over serial.

1 Domain Analysis

1.1 Finite-State Machines in Embedded Systems

A finite-state machine represents system behaviour as a limited number of states, transitions, inputs, and outputs. This approach is useful in embedded systems because it separates behaviour from low-level hardware access and makes transitions easy to verify.

The Button-LED problem is naturally represented as a Moore-style FSM. The output depends only on the current state: in the OFF state the LED output is 0, and in the ON state the LED output is 1. The input event is a valid debounced button press, which toggles the state.

1.2 Hardware Components and Justification

Component	Qty	Role in the application
Arduino Uno (ATmega328P)	1	Main MCU for button sampling, debounce, FSM evaluation, LED output, LCD updates, and serial reporting.
Pushbutton	1	External event source that requests a state transition.
LED	1	Visual output controlled by the FSM.
220 Ω resistor	1	Current-limiting resistor for the LED.
LCD1602	1	Local state display.
USB serial terminal	1	Receives <code>printf()</code> state reports.

Table 1: Hardware components used in Lab 11 Part 1

1.3 Software Components and Technologies

- **PlatformIO**: project build and firmware generation.
- **Arduino framework**: GPIO, timing, serial, and LCD support.
- **AVR-libc STDIO**: redirects `printf()` to serial output.
- **LiquidCrystal**: drives the 16x2 LCD in 4-bit mode.
- **PlantUML**: generates the architectural and behavioural diagrams used in this report.

2 Conceptualisation and Design

2.1 Technical Requirements

ID	Category	Requirement	Type
R-F-01	FSM	System shall implement exactly two states: OFF and ON.	Func.
R-F-02	Transition	System shall toggle state on each valid button press.	Func.
R-F-03	Debounce	System shall debounce the button before accepting a transition.	Func.
R-F-04	Output	LED shall reflect the current FSM state.	Func.
R-F-05	Reporting	State shall be shown through serial <code>printf()</code> and LCD.	Func.
R-NF-01	Latency	System shall respond to a stable press within 100 ms.	Non-func.
R-NF-02	Modularity	FSM behaviour shall be separated into an application module.	Non-func.
R-NF-03	Platform	Firmware shall fit Arduino Uno memory limits.	Non-func.

Table 2: Technical requirements for Lab 11 Part 1

2.2 FSM Definition

The FSM has two Moore states. The output is defined only by the current state:

State	LED Output	No valid press	Valid press
OFF	0	OFF	ON
ON	1	ON	OFF

Table 3: Button-LED finite-state transition table

2.3 Architectural and Behavioural Diagrams

3 Hardware and Software Implementation

3.1 Hardware Setup

Pin assignment:

- D9 – FSM LED through $220\ \Omega$ resistor
- D12 – pushbutton input using `INPUT_PULLUP`

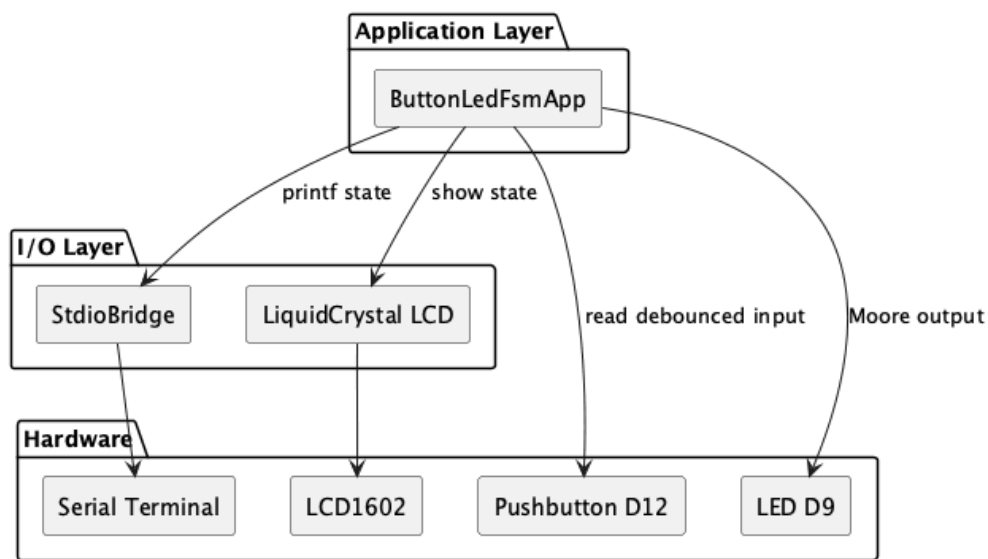


Figure 1: Layered architecture and hardware/software interactions

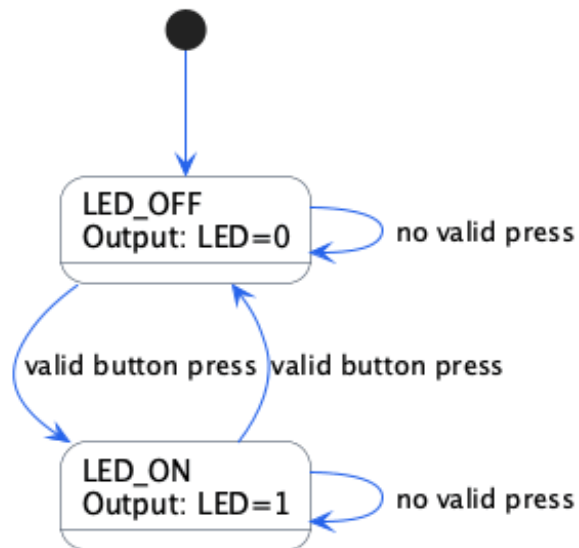


Figure 2: Finite-state machine for Button-LED control

- D7 – LCD RS
- D6 – LCD E
- D5 – LCD D4
- D4 – LCD D5
- D3 – LCD D6
- D13 – LCD D7
- D0/D1 – USB serial UART for STDIO output

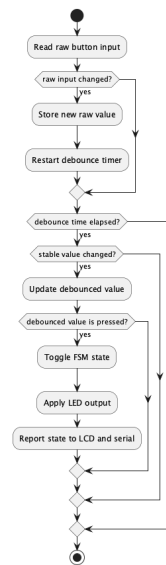


Figure 3: Debounce and valid-press processing flow

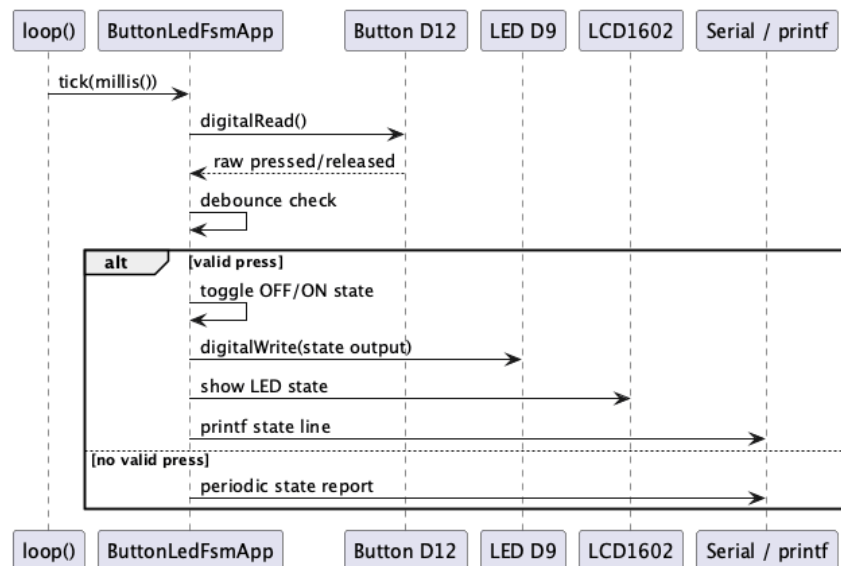


Figure 4: Sequence diagram for one application tick

3.2 Project Directory Structure

```

lab11/
|-- platformio.ini
|-- diagram.json
|-- wokwi.toml
|-- src/
|   |-- main.cpp
|   |-- config/
|   |   '-- AppConfig.h
|   |-- io/

```

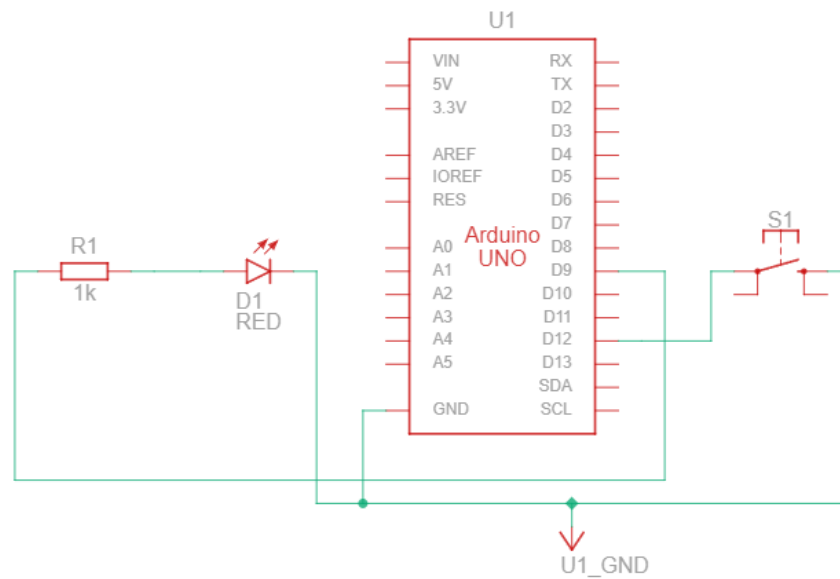


Figure 5: Electrical circuit schematic for Lab 11 Part 1

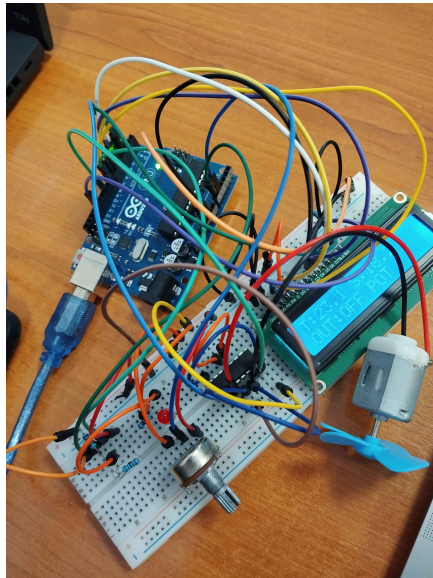


Figure 6: Hardware setup used for Lab 11 Part 1

```
|  |  |-- StdioBridge.h
|  |  '-- StdioBridge.cpp
|  '-- app/
|      |-- ButtonLedFsmApp.h
|      '-- ButtonLedFsmApp.cpp
'-- report/
    |-- main.tex
    |-- hardware_photo.jpg
    '-- plantuml/
```

3.3 Software Module Descriptions

- **ButtonLedFsmApp**: owns the FSM state, debounces the button, toggles state, drives the LED, updates the LCD, and reports through serial.
- **StdioBridge**: binds `printf()` to the Arduino serial port using AVR-libc.
- **AppConfig**: stores pin assignments and timing constants.
- **main.cpp**: initializes STDIO and delegates periodic execution to the app module.

4 Testing and Validation

4.1 Build Results

Environment	Flash used	Flash %	RAM used	RAM %
uno	5090 B	15.8%	451 B	22.0%

Table 4: PlatformIO build resource usage for Lab 11 Part 1

4.2 Test Scenarios

ID	Scenario	Observed result	Result
T-01	Startup	Firmware starts in OFF state and reports initial state.	PASS
T-02	First valid press	LED changes from OFF to ON after debounce.	PASS
T-03	Second valid press	LED changes from ON to OFF after debounce.	PASS
T-04	Button bounce	Fast unstable input changes do not create multiple transitions.	PASS
T-05	LCD output	LCD displays current LED state.	PASS
T-06	Serial output	Serial terminal receives state reports through <code>printf()</code> .	PASS
T-07	Memory limits	Flash and RAM usage remain within Uno limits.	PASS

Table 5: Validation scenarios for Button-LED FSM

4.3 Representative Serial Output

Lab11 Part 1 - Button LED finite-state machine

Pins: LED=D9 BUTTON=D12 (INPUT_PULLUP)

Button wiring: D12 -> button -> GND

```
t=0 state=OFF led=0 button=0
t=1240 state=ON led=1 button=1
t=2460 state=OFF led=0 button=1
```

General Conclusions

Laboratory Work #11 Part 1 was completed as a finite-state embedded application. The implemented Moore-style FSM has two states, OFF and ON, and toggles only on valid debounced button presses. The current state is visible both locally on LCD and remotely through `printf()` serial output.

Key outcomes:

1. The FSM structure makes the LED behaviour explicit and easy to validate.
2. Debouncing prevents false transitions caused by mechanical contact bounce.
3. STDIO and LCD reporting provide real-time observability of the current state.
4. The modular implementation fits comfortably within Arduino Uno resource limits.

Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code completion and refactoring assistance.
- **LLM assistance** – LaTeX drafting, editing support, and technical structure refinement.

All generated content was manually reviewed and adapted to the implemented project.

Bibliography

1. Arduino Official Reference. <https://www.arduino.cc/reference/en/>
2. AVR-libc User Manual. <https://www.nongnu.org/avr-libc/user-manual/>
3. PlatformIO Documentation. <https://docs.platformio.org/en/latest/>
4. PlantUML Documentation. <https://plantuml.com/>
5. Lab 11 condition document: Control comportamental cu automate finite, Part 1 Button-LED.

Annex – Source Code and PlantUML Sources

Repository URL:

<https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab11>

PlantUML source files are stored in:

`lab11/report/plantuml/`